widget-sdk

未发布作品。© 2023 Siemens 本材料包含 Siemens Industry Software, Inc. 及其子公司或关联公司（统称为"Siemens"）或其许可方的商业秘密或其他机密信息。对本信息的访问和使用严格受限于客户与 Siemens 签订的适用协议中的规定。未经 Siemens 明确书面许可，不得将本材料复制、分发或以其他方式在客户设施之外披露，也不得以 Siemens 未明确授权的任何方式使用。本文件仅供信息和指导之用。Siemens 保留随时更改本出版物中规格和其他信息的权利，恕不另行通知，读者在所有情况下应向 Siemens 咨询以确定是否有任何变更。本文件中关于产品、功能或特性的陈述构成技术信息，不构成保证或担保，不应对 Siemens 产生任何责任。Siemens 不承担所有保证，包括但不限于适销性和特定用途适用性的默示保证。特别是，Siemens 不保证产品的运行不会中断或无错误。 Siemens 产品销售和许可的条款和条件在 Siemens 与其客户的书面协议中规定。Siemens 的最终用户许可协议和通用合同协议可在以下网址查看：https://www.sw.siemens.com/en-US/sw-terms/ 商标：本文使用的商标、标志和服务标志（"Marks"）为 Siemens 或其他方的财产。未经 Siemens 或相关商标所有者的书面同意，任何人不得使用这些 Marks。本文中使用第三方 Marks 并非试图将 Siemens 表示为产品的来源，而是旨在表示来自特定第三方或与之相关的产品。Siemens 的商标列表可在以下网址查看： https://www.plm.automation.siemens.com/global/en/legal/trademarks.html。 注册商标 Linux® 根据 LMI（Linus Torvalds 的全球独家被许可方，该商标的所有者）的分许可使用。

关于 Siemens Digital Industries Software Siemens Digital Industries Software 是全球领先的产品生命周期管理（PLM）软件和服务提供商，拥有 700 万个授权席位和全球 71,000 家客户。总部位于德克萨斯州普莱诺，Siemens Digital Industries Software 与企业协作，提供开放的解决方案，帮助他们将更多想法转化为成功的产品。有关 Siemens Digital Industries Software 产品和服务的更多信息，请访问 https://www.siemens.com/plm 支持中心：https://www.support.sw.siemens.com 文档反馈：https://www.support.sw.siemens.com/doc_feedback_form

---

目录

1 Java Widget（小部件）示例 2 Velocity Widget 示例 3 Velocity 宏示例 4 在 Script Widget 中自定义现有 Widget 5 扩展 Velocity 上下文示例 6 小示例 7 页面脚本（Page Script） 8 Widget 参数之间的依赖关系 9 Widget 中的操作（Actions） 10 Velocity 上下文参考 11 在 Velocity 中使用 ParameterFactory 的示例 12 预定义的 Velocity 宏 13 在自定义 Widget 中启用 Work Item 属性侧边栏 14 自定义看板（Kanban）面板 15 自定义快速创建（Quick Create）对话框的内容 16 常见问题（F.A.Q）

---

1 Java Widget 示例

1.1 简介

本示例将详细介绍如何为 Pages 创建自定义 Java Widget（小部件）。

可扩展的内容： 通过开发自己的 Widget，可以扩展 Pages 中的 Widget 库。

示例将展示的内容： 如何创建自定义 Widget。将实现一个工作记录报告（Work Record Report）Widget，该 Widget 根据其父级和链接角色定义的项目计算工作记录。

1.2 Java API 工作空间准备

请参阅主 Polarion SDK 指南中的"工作空间准备"章节。

1.3 创建项目插件

创建项目插件的最佳方式是导入提供的示例。（它包含创建自定义插件所需的所有依赖项。）

1.3.1 导入示例

> **提示：** 确保您的插件是针对您使用的 Polarion 版本编译的。本示例包含一个预编译的 jar 插件。在开始基于本示例开发自己的插件之前，您可以将其删除。Eclipse 确保新的 jar 插件将根据您的源代码和 Polarion 版本创建。

选择 **文件 > 导入...** 1. 在出现的对话框中，在"常规"部分选择"现有项目到工作空间"，然后单击"下一步"。 2. 单击"浏览"，选择包含示例的目录。（通常在 `C:\Polarion\polarion\SDK\examples\` 。） 3. 提交。 4. 选择 `com.polarion.example.widget` ，然后单击"完成"。

1.4 部署到已安装的 Polarion

请参阅主 Polarion SDK 指南中的"部署到已安装的 Polarion"章节。

1.5 从工作空间执行

请参阅主 Polarion SDK 指南中的"从工作空间执行"章节。

1.6 配置

成功将插件部署到 Polarion 后，Work Record Widget 即可在 Pages 中使用。

> **提示：** 自定义 Widget 插件使用 HiveMind 进行部署。详情请参阅 `/src/META-INF/hivemodule.xml` 。部署配置位于 `/com.polarion.alm.ui/ui.jar/META-INF/richpages-hivemodule.xml` 。

---

2 Velocity Widget 示例

2.1 简介

本示例将详细介绍如何为 Pages 创建基于 Velocity 的自定义 Widget。

可扩展的内容： 通过开发自己的 Widget，可以扩展 Pages 的 Widget 库。

示例将展示的内容： 如何创建自定义 Widget。将实现一个工作记录报告 Widget，该 Widget 根据其父级和链接角色定义的项目计算工作记录。

2.2 部署

2.2.1 导入示例

使用您常用的 SVN 客户端。 1. 将 `com.polarion.example.velocitywidget` 的内容（通常在 `C:\Polarion\polarion\SDK\examples\` ）提交到仓库中的 `.polarion/pages/widgets` 。 2. 现在可以从页面设计器（即处于编辑模式的 Page）的 Widget 侧边栏中使用新 Widget。 3.

> **提示：** Velocity Widget 可以部署到仓库的全局（Global）和项目（Project）作用域。部署到全局作用域的 Widget 对所有项目可见。如果在全局和项目作用域中部署了两个具有相同 ID 的 Widget，则将使用项目作用域中的 Widget。

2.3 标签（Tags）

Widget 的标签（或分类）在 `widget.properties` 文件的 `tags` 属性中以逗号分隔的列表形式指定，可以是任何字符串或本地化键。有关标准标签的本地化键，请参阅 `com.polarion.alm.shared.api.model.rp.widget.RichPageWidget.getTags(SharedContext)` 的 javadoc。

---

3 Velocity 宏示例

3.1 简介

本示例将详细介绍如何为 Pages 部署自定义宏。

可扩展的内容： 通过开发自己的宏，可以扩展 Pages 的宏库。

示例将展示的内容： 如何创建自定义宏。将实现一个自定义信息框（Info box），用于显示信息。

3.2 部署

3.2.1 导入示例

使用您常用的 SVN 客户端。 1. 将 `com.polarion.example.velocitymacro` 的内容（通常在 `C:\Polarion\polarion\SDK\examples\` ）提交到仓库中的 `.polarion/pages/scripts/velocity` 。 2. 现在可以从 Script - Block Widget 和 Script - Inline Widget 中使用新宏。 3.

3.2.2 使用方法

1. 通过 Widget 侧边栏将 Script - Block Widget 或 Script - Inline Widget 放置到您的页面中。
2. 使用 `#import("macros.vm")` 导入宏。
3. 使用 `#infoBox("your message")` 来使用 infoBox 宏。

> **提示：** Polarion 首先在项目作用域中搜索包含的文件。如果未找到，则从全局作用域中获取。

---

4 在 Script Widget 中自定义现有 Widget

4.1 简介

本示例将详细介绍如何使用"复制 Widget 源代码"操作在 Script Widget 中自定义现有 Widget。

示例将展示的内容： 如何自定义现有的多行趋势图（Multi Line Trend Chart）Widget。（将显示按类别在时间轴上分开的工作项数量。）

4.2 实现

1. 将多行趋势图 Widget 插入到页面中。
2. 更改 Widget 的标题并单击"应用"。（这样所有参数都会被复制到 Widget 的源代码中。后续步骤需要这些参数。）
3. 单击 Widget 标题中的操作菜单按钮，选择"复制 Widget 源代码"。
4. 插入一个 Script Block Widget，将复制的源代码粘贴到打开的参数侧边栏中的"脚本"参数中。（您将看到带有预填充参数的基本 Widget 定义。）
5. 修改源代码使其与下面所示的代码一致。（唯一的变化应该是第一行的 Velocity 赋值和 `<sub id="elements">` 元素的内容。）

```
#set($projectId = "PROJECT_ID")
<div class="polarion-rp-widget-part" data-widget="com.polarion.multiSetTrendChart">
  <span class="polarion-rp-widget-parameters">
    <sub id="title">Work Items count separated by category</sub>
    <sub id="data">
      <sub id="elements">
        #set($categories = $transaction.categories().search().query("project.id:
$projectId"))
        #foreach ($category in  $categories)
          <sub>
            <sub id="prototype">WorkItem</sub>
            <sub id="queryType">lucene</sub>
            <sub id="luceneQuery">categories.id:$category.fields.id.get</sub>
            <sub id="children">
              <sub id="name">$category.fields.name.get</sub><sub id="color"></sub>
            </sub>
          </sub>
        #end
      </sub>
    </sub>
    <sub id="timeAxis">
      <sub id="from">
        <sub id="relative">-30</sub>
      </sub>
      <sub id="to">
        <sub id="relative">0</sub>
      </sub>
      <sub id="scaleType">week</sub>
      <sub id="baseDate">1</sub>
    </sub>
  </span>
</div>
```

1. 更改 `projectId` 变量，使其指向您选择的项目。
2. 单击"应用"。

> **提示：** Script Widget 有两种类型——Inline（行内）和 Block（块级）。Inline Widget 可以在段落内部使用，因为它们不会打断文本。Block Widget 用作独立组件。

5 扩展 Velocity 上下文示例

5.1 简介

本示例将详细介绍如何在脚本、Widget 和宏中部署和使用您自己的 HiveMind 服务或 Java 实例。

可扩展的内容： 通过开发自己的服务和对象，可以扩展 Pages、Widget、宏和支持它的参数中使用的 Velocity 上下文。

示例将展示的内容： 如何扩展 Velocity 上下文。将实现 Work Item Util 并作为 Java 实例贡献，用于计算给定 Work Item 上花费的时间。然后将实现 Repository Util 并作为 HiveMind 服务贡献，提供获取仓库中最新修订版的方法，并提供 IRepositoryService。

5.2 Java API 工作空间准备

请参阅主 Polarion SDK 指南中的"工作空间准备"章节。

5.3 创建项目插件

创建项目插件的最佳方式是导入提供的示例，该示例包含创建自定义插件所需的所有依赖项。

5.3.1 导入示例

> **提示：** 确保您的插件是针对您使用的 Polarion 版本编译的。本示例包含一个预编译的 jar 插件。在开始基于本示例开发插件之前，您可以将其删除。Eclipse 确保新的 jar 插件将根据您的源代码和 Polarion 版本创建。

选择 **文件 > 导入...** 1. 在出现的对话框中，在"常规"部分选择"现有项目到工作空间"，然后单击"下一步"。 2. 单击"浏览"，选择示例目录。（通常在 `C:\Polarion\polarion\SDK\examples\` 。） 3. 提交。 4. 选择 `com.polarion.example.velocitycontext` ，然后单击"完成"。

5.4 部署到已安装的 Polarion

请参阅主 Polarion SDK 指南中的"部署到已安装的 Polarion"章节。

5.5 从工作空间执行

请参阅主 Polarion SDK 指南中的"从工作空间执行"章节。

5.6 配置

成功将插件部署到 Polarion 后，您可以在 Pages 上的 Velocity 代码中开始使用 Work Item Util 和 Repository Util：

- `$workItemUtil` - Work Item Util
- `$repositoryUtil` - Repository Util

5.6.1 Work Item Util 使用示例

创建一个页面，例如包含一个 Script Block Widget。使用以下内容作为脚本内容：

```
$workItemUtil.printTimeSpent($transaction.workItems().getBy().ids("PROJECT_ID",
"WORKITEM_ID"))
```

将 `PROJECT_ID` 和 `WORKITEM_ID` 替换为具有工作记录的项目和工作项 ID。

> **提示：** Velocity 上下文插件使用 HiveMind 进行部署。详情请参阅 `/src/META-INF/hivemodule.xml`。

5.7 创建 Inline Widget

要启用 Inline Widget，请在目标 Widget 的文件夹中创建 `widget.properties` 文件并添加以下行：

```
inline=true
```

---

6 小示例

6.1 创建自定义 Highcharts 图表

本示例详细介绍如何使用 Script Block Widget 通过 Velocity 和 Polarion 渲染 API 生成自定义 Highcharts 图表。

```
#set($html = $renderingContext.createHtmlFragmentBuilder())
#set($chart = $renderingContext.createChartBuilder())
#set($pie = $chart.pie().name("Number of demands"))
$!pie.addValue("yksi", 1).null
$!pie.addValue("kaksi", 2).null
$!pie.addValue("kolme", $math.toDouble("3.3")).null
$!chart.build().title().text("Sample").null
$!chart.build().plotOptions().pie().addRawAttribute("tooltip", "{pointFormat:
'<b>{point.y}</b>'}").null
$!chart.build().render($html, 300, $renderingContext.columnWidth).null
$html
```

示例输出：

> Sample yksi | kaksi | kolme（饼图）

6.2 为 Highcharts 图表设置原始属性

本示例展示如何为这些图表设置原始属性。

```
#set($html = $renderingContext.createHtmlFragmentBuilder())
#set($builder = $renderingContext.createChartBuilder().build())
$!builder.title().text("Chart").null
$!builder.chart().type().line().null
$!builder.addRawAttribute("noData","{style: {fontWeight: 'bold', fontSize: '15px',
color: '#FF0000'}}").null
$!builder.render($html, 300, $renderingContext.columnWidth).null
$html
```

> No data to display（无数据显示） Chart（图表标题）

Highcharts 现在已配置完成，您可以使用 Velocity 从 Polarion 中提取数据来填充图表。

6.3 将一组对象渲染为表格

以下代码示例展示如何使用 Script Block Widget 渲染一个显示对象属性的表格。对象集由查询定义。

示例 1：

```
<table class="polarion-rpw-table-content">
  <tbody>
    <tr class="polarion-rpw-table-header-row">
      <th>Item</th>
      <th>Status</th>
      <th>Parent</th>
    </tr>
#foreach($w in $transaction.workItems.search.query("project.id:drivepilot"))
    <tr class="polarion-rpw-table-content-row">
      <td>$w.render.withLinks.withTitle</td>
      <td>$w.fields.status.render</td>
      <td>
        #foreach($l in $w.fields.linkedWorkItems.direct)
          #if($l.fields.role.get.id == "parent")
            $l.fields.workItem.render.withTitle.withLinks
          #end
        #end
      </td>
    </tr>
#end
  </tbody>
</table>
```

上面的代码渲染出以下表格：

Item	Status	Parent
SDK-2 - Child Requirement	Open	SDK-1 - Parent Requirement
SDK-1 - Parent Requirement	Open	

示例 2：

```
<table class="polarion-rpw-table-content">
  <tbody>
    <tr class="polarion-rpw-table-header-row">
      <th>Plan #</th>
      <th># of Work Items at Start</th>
      <th># of WIs at End</th>
    </tr>
#set($counter=1)
#foreach($plan in $transaction.plans().search().limit(19))
  #if($plan.fields().startedOn().get() && $plan.fields().finishedOn().get() &&
$plan.fields().name().get().contains("Team"))
    #set($revisionStarted =
$transaction.revisions().getBy().dateOrAfter($plan.fields().startedOn().get()).fiel
ds().id().get())
    #set($planAtStart =
$transaction.plans().getBy().revision($revisionStarted).path($plan.getReference().t
oPath()))
    #set($revisionFinished =
$transaction.revisions().getBy().dateOrAfter($plan.fields().finishedOn().get()).fie
lds().id().get())
    #set($planAtEnd =
$transaction.plans().getBy().revision($revisionFinished).path($plan.getReference().
toPath()))
    #if(!$planAtStart.isUnresolvable() && !$planAtEnd.isUnresolvable())
    <tr class="polarion-rpw-table-content-row">
      <td>$counter</td>
      <td>$planAtStart.items().size()</td>
      <td>$planAtEnd.items().size()</td>
    </tr>

    #set($counter=$math.add($counter, 1))
    #end
  #end
#end
  </tbody>
</table>
```

6.4 用于选择字段的 Widget 参数

本示例展示如何使用字段选择器 Widget 参数以及如何定义默认选中的字段。

这是 Widget 的 `parameters.vm` 文件内容：

```
#set($myFields = $objectFactory.newSet())
#set($void = $myFields.add("priority"))
#set($void = $myFields.add("severity"))
$parameters.put("myFieldsParameter", $factory.fields("My Fields
Parameter").fields($myFields).build())
```

6.5 按日期获取历史对象

以下代码示例详细介绍如何获取给定日期的对应修订版。（接着是如何使用该修订版获取对象的历史数据。）

示例显示了计划开始和结束时链接到计划的工作项数量。

```
<table class="polarion-rpw-table-content">
  <tbody>
    <tr class="polarion-rpw-table-header-row">
      <th>Plan #</th>
      <th># of Work Items at Start</th>
      <th># of WIs at End</th>
    </tr>
#foreach($plan in $transaction.plans().search())
  #set($revisionStarted =
$transaction.revisions().getBy().dateOrAfter($plan.fields().startedOn().get()).fiel
ds().id().get())
  #set($planAtStart =
$transaction.plans().getBy().revision($revisionStarted).path($plan.getReference().t
oPath()))
  #set($revisionFinished =
$transaction.revisions().getBy().dateOrAfter($plan.fields().finishedOn().get()).fie
lds().id().get())
  #set($planAtEnd =
$transaction.plans().getBy().revision($revisionFinished).path($plan.getReference().
toPath()))
    <tr class="polarion-rpw-table-content-row">
      <td>$plan.fields().name().get()</td>
      <td>$planAtStart.items().size()</td>
      <td>$planAtEnd.items().size()</td>
    </tr>

#end
  </tbody>
</table>
```

获取修订版的历史数据：

```
<table class="polarion-rpw-table-content">
  <tbody>
    <tr class="polarion-rpw-table-header-row">
      <th>Plan #</th>
      <th># of Work Items at Start</th>
      <th># of WIs at End</th>
    </tr>
#set($counter=1)
#foreach($plan in $transaction.plans().search().limit(19))
  #if($plan.fields().startedOn().get() && $plan.fields().finishedOn().get() &&
$plan.fields().name().get().contains("Team"))
    #set($revisionStarted =
$transaction.revisions().getBy().dateOrAfter($plan.fields().startedOn().get()).fiel
ds().id().get())
    #set($planAtStart =
$transaction.plans().getBy().revision($revisionStarted).path($plan.getReference().t
oPath()))
    #set($revisionFinished =
$transaction.revisions().getBy().dateOrAfter($plan.fields().finishedOn().get()).fie
lds().id().get())
    #set($planAtEnd =
$transaction.plans().getBy().revision($revisionFinished).path($plan.getReference().
toPath()))
    #if(!$planAtStart.isUnresolvable() && !$planAtEnd.isUnresolvable())
    <tr class="polarion-rpw-table-content-row">
      <td>$counter</td>
      <td>$planAtStart.items().size()</td>
      <td>$planAtEnd.items().size()</td>
    </tr>

    #set($counter=$math.add($counter, 1))
    #end
  #end
#end
  </tbody>
</table>
```

6.6 如何在 Velocity 和查询中使用页面参数（Page Parameters）

6.6.1 字符串参数（String Parameters）

以下示例引用了一个页面参数，其 ID 为"Name"，类型为"String"，值为"John Doe"。

访问实例：

```
$pageParameters.Name => StringParameterImpl: Name, John Doe
```

获取值：

```
$pageParameters.Name.value => John Doe
```

获取可在 SQL 查询中使用的值：

```
$pageParameters.Name.toSql => John Doe
```

获取可在 Lucene 查询中使用的值：

```
$pageParameters.Name.toLucene => "John Doe"
```

6.6.2 整数参数（Integer Parameters）

以下示例引用了一个页面参数，其 ID 为"Year"，类型为"Integer"，值为"2014"。

访问实例：

```
$pageParameters.Year => IntegerParameterImpl: Year, 2014
```

获取值：

```
$pageParameters.Year.value => 2014
```

获取可在 SQL 查询中使用的值：

```
$pageParameters.Year.toSql => 2014
```

获取可在 Lucene 查询中使用的值：

```
$pageParameters.Year.toLucene => 2014
```

6.6.3 布尔参数（Boolean Parameters）

以下示例引用了一个页面参数，其 ID 为"Advanced"，类型为"Boolean"，值为"Yes"。

访问实例：

```
$pageParameters.Advanced => BooleanParameterImpl: Advanced, true
```

获取值：

```
$pageParameters.Advanced.value => true
```

获取可在 SQL 查询中使用的值：

```
$pageParameters.Advanced.toSql => true
```

获取可在 Lucene 查询中使用的值：

```
$pageParameters.Advanced.toLucene => true
```

6.6.4 枚举参数（Enumeration Parameters）

以下示例引用了一个页面参数，其 ID 为"reqtype"，类型为"Enumeration"，值为"Functional"和"Business"。

访问实例：

```
$pageParameters.reqtype => EnumParameterImpl: Requirement Types, [functional,
business]
```

获取值列表：

```
$pageParameters.reqtype.values => [ProxyEnumOption id:functional, ProxyEnumOption
id:business]
```

返回的列表可在 `#foreach` 循环中使用。即使枚举不是多值的，也会返回列表。

获取第一个值（或单值枚举的唯一值）：

```
$pageParameters.reqtype.singleValue => ProxyEnumOption id:functional
```

获取可在 SQL 查询中使用的值：

```
$pageParameters.reqtype.toSql => ('functional', 'business')
```

使用示例：`WHERE C_TYPE IN $pageParameters.reqtype.toSql`

获取可在 Lucene 查询中使用的值：

```
$pageParameters.reqtype.toLucene => (functional business)
```

使用示例：`reqtype:$pageParameters.reqtype.toLucene`

6.6.5 自定义枚举参数（Custom Enumeration Parameters）

以下示例引用了一个页面参数，其 ID 为"color"，类型为"Custom Enumeration"，名称为"Red"、"Green"和"Blue"，值为"#f00"、"#0f0"和"#00f"。

访问实例：

```
$pageParameters.color =>
com.polarion.alm.shared.api.model.rp.parameter.impl.enumeration.CustomEnumParameter
Impl@f74eaf2
```

获取值列表：

```
$pageParameters.color.values => [#f00, #0f0, #00f]
```

返回的列表可在 `#foreach` 循环中使用。即使枚举不是多值的，也会返回列表。

获取第一个值（或单值枚举的唯一值）：

```
$pageParameters.color.singleValue => #f00
```

获取名称列表：

```
$pageParameters.color.names => [Red, Green, Blue]
```

返回的列表可在 `#foreach` 循环中使用。即使枚举不是多值的，也会返回列表。

获取第一个名称（或单值枚举的唯一名称）：

```
$pageParameters.color.singleName => Red
```

6.6.6 日期参数（Date Parameters）

以下示例引用了一个页面参数，其 ID 为"From"，类型为"Date"，绝对日期为"2015年3月20日"。

访问实例：

```
$pageParameters.From => DateParameterImpl: From, Fri Mar 20 00:00:00 CET 2015
```

获取值：

```
$pageParameters.From.value => Fri Mar 20 00:00:00 CET 2015
```

获取可在 SQL 查询中使用的值：

```
$pageParameters.From.toSql => 2015-03-20
```

获取可在 Lucene 查询中使用的值：

```
$pageParameters.From.toLucene => 20150320
```

6.6.7 在"Lucene + Velocity"和"SQL + Velocity"查询中的使用

当页面参数在"Lucene + Velocity"和"SQL + Velocity"查询中使用时，`toSql()` 和 `toLucene()` 方法调用会自动添加。因此，给定一个 ID 为"Year"的页面参数，您可以使用以下查询获取该年所有已解决的项目：

```
resolvedOn:[${pageParameters.Year}0101 TO ${pageParameters.Year}1231]
```

更多信息请参阅 `com.polarion.alm.shared.api.model.rp.parameter.ParameterUsableInQuery` 接口定义的方法的 Javadoc。

---

7 页面脚本（Page Script）

页面脚本在 Widget 之前执行，可用于： - 初始化变量，供后续 Widget 使用。 - 添加页面参数，Widget 参数可以绑定到这些参数。

页面脚本可以使用与 Widget 相同的 Velocity 上下文变量，但有以下例外： - `parameters` 不可用。 - `pageContext` 可用。（它是上下文数据的持有者。） - `factory` 可用于创建脚本化页面参数。 - `scriptedPageParameters` 可用于添加脚本化页面参数。

执行页面脚本的结果可以通过侧边栏工具栏中的"显示结果"操作来显示。（对调试很有用。）

Velocity 记录的错误和警告可以通过侧边栏菜单中的"显示问题"操作来显示。（仅在报告问题时显示。）

7.1 页面上下文（Page Context）

页面上下文是页面脚本准备的变量的持有者，然后由脚本化 Widget 访问。它只能由页面脚本修改。

Widget 的渲染在某些情况下是并发运行的，因此如果页面上下文中的变量被页面上的多个 Widget 访问，它们必须是线程安全的对象。

从事务返回的 API 对象（如 WorkItem 或 Document）不应存储到页面上下文中，因为它们只能在给定的事务中使用，而 Widget 的渲染有时在不同的事务中执行。相反，应存储引用（如通过 `WorkItem.getReference()` 方法获取的 WorkItemReference），然后在 Widget 中使用 `$reference.get($transaction)` 来重新获取 WorkItem。

7.1.1 页面上下文示例

以下页面脚本是一个示例，展示如何准备 SQL 查询的公共部分，供多个 Widget 以不同方式显示相同数据：

```
#set($uncoveredReqWhere = $objectFactory.newStringBuilder)
$!uncoveredReqWhere.append("and C_TYPE in $pageParameters.reqtypes.toSql $
{esc.n}").null
#if(!$pageParameters.version.values.empty)
    $!uncoveredReqWhere.append("and CUSTOM_FIELD.FK_WORKITEM = WORKITEM.C_PK $
{esc.n}").null
    $!uncoveredReqWhere.append("and CUSTOM_FIELD.C_NAME = 'targetVersion' $
{esc.n}").null
    $!uncoveredReqWhere.append("and CUSTOM_FIELD.C_STRING_VALUE =
'$pageParameters.version.singleValue().id()' ${esc.n}").null
#end
#if(!$pageParameters.status.values.empty)
    $!uncoveredReqWhere.append("and C_STATUS IN $pageParameters.status.toSql $
{esc.n}").null
#end
#set($coveredReqWhere = $objectFactory.newStringBuilder)
$!coveredReqWhere.append($uncoveredReqWhere).null
```

```
#set($testQuery = $objectFactory.newStringBuilder)
$!testQuery.append("select TEST.C_PK from WORKITEM TEST,
STRUCT_WORKITEM_LINKEDWORKITEMS LINK ${esc.n}").null
$!testQuery.append("    where LINK.FK_WORKITEM = WORKITEM.C_PK ${esc.n}").null
$!testQuery.append("    and LINK.FK_P_WORKITEM = TEST.C_PK ${esc.n}").null
$!testQuery.append("    and LINK.C_ROLE =
'$pageParameters.verifiesLinkRole.toSql'").null
$!uncoveredReqWhere.append("and not exists ($!testQuery)").null
$!coveredReqWhere.append("and exists ($!testQuery)").null
$!pageContext.put("uncoveredReqWhere", "$uncoveredReqWhere")
$!pageContext.put("coveredReqWhere", "$coveredReqWhere")
#if(!$pageParameters.version.values.empty)
    $!pageContext.put("joinCustomField", ",CF_WORKITEM CUSTOM_FIELD").null
#end
## 在结果中显示值用于调试
<pre>
uncoveredReqWhere:
$uncoveredReqWhere
coveredReqWhere:
$coveredReqWhere
</pre>
```

该示例是为 Drive Pilot 演示项目开发的，使用了以下页面参数： - `reqtypes` - 需求项类型 - 工作项类型枚举的多枚举参数。 - `version` - 版本 - 查询为 `template.id:release` 的计划枚举参数。 - `status` - 状态 - 状态枚举的枚举参数。

此页面脚本的结果：

```
uncoveredReqWhere:
and C_TYPE in ('systemRequirement', 'softwareRequirement', 'mechanicalRequirement',
'electricalRequirement')
and CUSTOM_FIELD.FK_WORKITEM = WORKITEM.C_PK
and CUSTOM_FIELD.C_NAME = 'targetVersion'
and CUSTOM_FIELD.C_STRING_VALUE = 'Version_2_0'
and C_STATUS IN ('approved')
and not exists (select TEST.C_PK from WORKITEM TEST,
STRUCT_WORKITEM_LINKEDWORKITEMS LINK
    where LINK.FK_WORKITEM = WORKITEM.C_PK
    and LINK.FK_P_WORKITEM = TEST.C_PK
    and LINK.C_ROLE = 'verifies')

coveredReqWhere:
and C_TYPE in ('systemRequirement', 'softwareRequirement', 'mechanicalRequirement',
'electricalRequirement')
and CUSTOM_FIELD.FK_WORKITEM = WORKITEM.C_PK
and CUSTOM_FIELD.C_NAME = 'targetVersion'
and CUSTOM_FIELD.C_STRING_VALUE = 'Version_2_0'
and C_STATUS IN ('approved')
and exists (select TEST.C_PK from WORKITEM TEST, STRUCT_WORKITEM_LINKEDWORKITEMS
LINK
    where LINK.FK_WORKITEM = WORKITEM.C_PK
    and LINK.FK_P_WORKITEM = TEST.C_PK
    and LINK.C_ROLE = 'verifies')
```

使用这些变量的示例 Widget。Widget 的参数显示在右侧：

7.2 脚本化页面参数（Scripted Page Parameters）

页面脚本可以在页面参数侧边栏中定义的页面参数之上添加额外的页面参数。页面脚本不能添加与侧边栏中已定义参数具有相同 ID 的页面参数，如果尝试这样做，将在页面脚本错误日志中记录错误。

7.2.1 脚本化页面参数示例

下面的页面脚本展示了如何添加 2 个日期类型的页面参数，其值取自计划的开始日期和截止日期字段。

该示例适用于计划报告中使用。

```
#set($startDate = $plan.fields.startDate.getIfCan)
#set($startDateBuilder = $factory.date("Start Date"))
#if ($!startDate)
    $!startDateBuilder.absolute($!startDate)
#end
$!scriptedPageParameters.put("startDate", $startDateBuilder.build())

#set($dueDate = $plan.fields.dueDate.getIfCan)
#set($dueDateBuilder = $factory.date("Due Date"))
#if ($!dueDate)
    $!dueDateBuilder.absolute($!dueDate)
#end
$!scriptedPageParameters.put("dueDate", $dueDateBuilder.build())
```

这些参数随后可用于 Widget 中的任何日期参数，例如在图表 Widget 中，如下所示：

---

8 Widget 参数之间的依赖关系

可以定义 Widget 参数之间的依赖关系。当报告设计器更改标记为"依赖源"的 Widget 参数的值时，任何标记为"依赖目标"的其他参数可以相应地重新配置。

8.1 示例

一个 Widget 有两个参数： - 第一个是对象选择器参数，用户可以在其中选择一个工作项（Work Item）。 - 第二个是枚举参数，用于选择链接角色。

第二个参数应提供从工作项中定义的项目中选择链接角色的可能性。

此示例的完整实现可在 Work Records（Java 和 Velocity）Widget 示例中找到。

8.1.1 Java Widget

参数定义：

```
parameters.put(PARAMETER_USER_STORY, factory.objectSelector("User
Story").allowedPrototypes(PrototypeEnum.WorkItem).dependencySource(true).build());
parameters.put(PARAMETER_LINK_ROLE, factory.enumeration("Link Role", "workitem-
link-role").dependencyTarget(true).build());
```

实现依赖方法： 应在 Widget 类中重写

`com.polarion.alm.shared.api.model.rp.widget.RichPageWidget.processParameterDependencies(RichPageWi`

```
ObjectSelectorParameter parameter =
context.parameter(WorkRecordReportWidget.PARAMETER_USER_STORY);
WorkItem workItem = (WorkItem) parameter.value();
Scope scope = (workItem == null) ? null : workItem.getReference().scope();
EnumParameter parameter =
context.parameter(WorkRecordReportWidget.PARAMETER_LINK_ROLE);
parameter.set().scope(scope);
```

8.1.2 Velocity Widget

参数定义：

```
$parameters.put("userStory", $factory.objectSelector("User
Story").allowedPrototypes(["WorkItem"]).dependencySource(true).build())
$parameters.put("linkRole", $factory.enumeration("Link Role", "workitem-link-
role").dependencyTarget(true).build())
```

依赖实现在 `dependencies.vm` 文件中定义。

```
#set($userStoryParameter = $parameters.userStory)
#set($linkRoleParameter = $parameters.linkRole)
#set($workItem = $userStoryParameter.value)
#if (!$workItem)
    #set($scope = $null)
```

```
#else
    #set($scope = $workItem.getReference().scope())
#end
$linkRoleParameter.set().scope($scope)
```

---

9 Widget 中的操作（Actions）

可以在 Widget 中创建一个按钮，单击时触发服务器端操作。（例如，启动计划的按钮。）

9.1 工作原理

渲染的 Widget 包含定义了以下特殊属性（`RichPageWidget.ATTRIBUTE_ACTION_ID`）的元素。

当用户单击它时，将向服务器发送请求并触发 `RichPageWidget.executeAction(RichPageWidgetActionContext)`。

Widget 开发人员可以通过在 Widget 中实现 `executeAction` 来处理该事件。

> **注意：** 标准操作处理会持久化一些更改，因此最好事先请求用户确认。（请参阅 `RichPageWidget.ATTRIBUTE_CONFIRM_*` 键。）

Widget 开发人员还可以定义操作执行后是刷新整个页面还是仅重新渲染 Widget。 这可以通过调用 `RichPageWidgetActionContext.refresh(boolean)` 来完成。

重要的是要理解，同一页面上可能有多个相同类型的 Widget，因此确定应执行的操作应基于 `ATTRIBUTE_ACTION_ID` 值和 Widget 参数。

操作处理仅适用于 Java Rich Page Widget。

可以通过 `ModelObjectBase.getUpdatable(WriteTransaction)` 获取模型对象的可更新版本。

9.2 示例

```
private void renderAction(@NotNull HtmlContentBuilder htmlContentBuilder, @NotNull
DurationValue total) {
    HtmlTagBuilder a = htmlContentBuilder.tag().a();
    a.append().text("Update time spent in the user story");
    a.attributes().byName(RichPageWidget.ATTRIBUTE_ACTION_ID, total.toString());
    a.attributes().byName(RichPageWidget.ATTRIBUTE_CONFIRM_TITLE, "Update time
spent");
    a.attributes().byName(RichPageWidget.ATTRIBUTE_CONFIRM_TEXT, "Do you want to
update time spent?");
}

public void executeAction(@NotNull RichPageWidgetActionContext context) {
    ObjectSelectorParameter parameter = context.parameter(PARAMETER_USER_STORY);
    WorkItem workItem = (WorkItem) parameter.value();
    if (workItem == null) {
        return;
    }
    Duration value;
    try {
        value = new Duration(context.actionId());
    } catch (Exception e) {
        return;
    }
    UpdatableWorkItem updatableWorkItem =
workItem.getUpdatable(context.transaction());
    updatableWorkItem.fields().timeSpent().set(value);
    updatableWorkItem.save();
    context.refresh(true);
}
```

此示例的完整实现可在 Java Work Records Widget 示例中找到。

---

10 Velocity 上下文参考

10.1 全局可用

10.1.1 Polarion 对象

变量	类型	说明
`transaction`	`com.polarion.alm.shared.api.transaction.ReadOnlyTransaction`	只读事务
`localization`	`com.polarion.alm.shared.api.utils.SharedLocalization`	本地化工具
`me`	`java.lang.String`	当前用户 ID（`$transaction` 的快捷方式）
`objectFactory`	`com.polarion.alm.shared.api.utils.velocity.ObjectFactory`	对象工厂

10.1.2 Polarion 服务（来自 Polarion Open Java API）

变量	类型
`trackerService`	`com.polarion.alm.tracker.ITrackerService`
`projectService`	`com.polarion.alm.projects.IProjectService`
`securityService`	`com.polarion.platform.security.ISecurityService`
`platformService`	`com.polarion.platform.IPlatformService`
`testManagementService`	`com.polarion.alm.tracker.ITestManagementService`

10.1.3 Velocity 通用工具

变量	说明
`date`	日期工具 - http://velocity.apache.org/tools/2.0/apidocs/org/apache/velocity/tools/generic/DateTool.html
`math`	数学工具 - http://velocity.apache.org/tools/2.0/apidocs/org/apache/velocity/tools/generic/MathTool.html
`number`	数字工具 - http://velocity.apache.org/tools/2.0/apidocs/org/apache/velocity/tools/generic/NumberTool.html

变量	说明
`esc`	转义工具 - http://velocity.apache.org/tools/2.0/apidocs/org/apache/velocity/tools/generic/EscapeTool.html
`alternator`	交替工具 - http://velocity.apache.org/tools/2.0/apidocs/org/apache/velocity/tools/generic/AlternatorTool.html
`lists`	列表工具 - http://velocity.apache.org/tools/2.0/apidocs/org/apache/velocity/tools/generic/ListTool.html
`sorter`	排序工具 - http://velocity.apache.org/tools/2.0/apidocs/org/apache/velocity/tools/generic/SortTool.html
`mill`	迭代器工具 - http://velocity.apache.org/tools/2.0/apidocs/org/apache/velocity/tools/generic/IteratorTool.html

10.2 在 Velocity Widget 和页面脚本中可用

以下所有内容在"Lucene + Velocity"和"SQL + Velocity"查询中也可用。

10.2.1 Velocity Widget 对象

变量	类型
`renderingContext`	页面脚本中为 `com.polarion.alm.shared.api.model.rp.widget.RichPageRenderingContext` 情况下为 `com.polarion.alm.shared.api.model.rp.widget.RichPageWidgetRenderingCo`
`widgetContext`	同 `renderingContext`
`pageContext`	`java.util.Map<Object, Object>`
`parameters`	`com.polarion.alm.shared.api.utils.collections.ReadOnlyStrictMap<Strin` `extends com.polarion.alm.shared.api.model.rp.parameter.RichPageParamet`
`pageParameters`	`com.polarion.alm.shared.api.utils.collections.ReadOnlyStrictMap<Strin` `extends com.polarion.alm.shared.api.model.rp.parameter.RichPageParamet`

变量	类型
`scriptedPageParameters`	`com.polarion.alm.shared.api.utils.collections.StrictMap<String, ? ext` `com.polarion.alm.shared.api.model.rp.parameter.RichPageParameter>`
`urlParameters`	`com.polarion.alm.shared.api.utils.collections.ReadOnlyStrictMap<Strin` `String>`
`page`	`com.polarion.alm.shared.api.model.rp.RichPage`
`plan`	`com.polarion.alm.shared.api.model.plan.Plan`
`testRun`	`com.polarion.alm.shared.api.model.tr.TestRun`
`object`	`com.polarion.alm.shared.api.model.ModelObject`
`factory`	`com.polarion.alm.shared.api.model.rp.parameter.ParameterFactory`

10.3 访问派生字段（Derived Fields）

Polarion 渲染 API 中的派生字段访问器以前缀下划线（`_`）开头。Velocity 解析器无法处理此类方法，但会接受不带前导下划线或括号的方法名称，例如：

```
$transaction.userGroups.getBy.id('someId').fields.users.size  ✓ 可以工作
$transaction.userGroups.getBy.id('someId').fields._users.size ✗ 不行
```

---

11 在 Velocity 中使用 ParameterFactory 的示例

11.1 在页面脚本中添加脚本化页面参数

```
$!scriptedPageParameters.put("myString", $factory.string("My String").value("my value").build())
$!scriptedPageParameters.put("myInteger", $factory.integer("My Integer").value(3).build())
$!scriptedPageParameters.put("myBoolean", $factory.bool("My Boolean").value(true).build())
$!scriptedPageParameters.put("mySingleSeverity", $factory.enumeration("My Single Severity", "severity").singleValue("major").build())
$!scriptedPageParameters.put("myMultiSeverity", $factory.enumeration("My Multi Severity", "severity").allowMultipleValues(true).values(["major", "critical"]).build())
$!scriptedPageParameters.put("myRelativeDate", $factory.date("My Relative Date").relative(-12).build())
$!scriptedPageParameters.put("myAbsoluteDate", $factory.date("My Absolute Date").absolute($date.toDate("yyyy-MM-dd", "2015-01-02")).build())
$!scriptedPageParameters.put("myPlan", $factory.objectSelector("My Plan").allowedPrototypes(["Plan"]).objectPath("drivepilot/Version_1_0").build())
```

其他参数类型不能用作页面参数。

11.2 在 parameters.vm 中添加 Widget 参数

上述所有示例均可使用。只需将 `$!scriptedPageParameters` 替换为 `$!parameters`。

其他参数类型示例：

```
$!parameters.put("myScript", $factory.script("My Script").value("${esc.d}
me").build())
$!parameters.put("myTimeAxis", $factory.timeAxis("My Time
Axis").fromRelative(-28).toRelative(0).scaleWeek().build())
$!parameters.put("myFields", $factory.fields("My
Fields").fields($mySetOfFields).build())
$!parameters.put("myQuery", $factory.luceneQuery("My
Query").scope().global().prototype().workItem().subtype("defect").value("HAS_VALUE:
resolution").build())
```

结合 DataSetParameter 与 FieldsParameter 和 SortingParameter 的示例。（Table Widget 中使用的方式。）

```
#set($myColumns = $factory.fields("My Columns").build())
#set($mySorting = $factory.sorting("My Sorting").build())
$!parameters.put("myDataSet", $factory.dataSet("My Data Set").add("myColumns",
$myColumns).add("mySorting", $mySorting).build())
```

自定义枚举参数的示例：

```
$parameters.put("teams", $factory.customEnum("Teams").addEnumItem("mainDev","Main
Development Team").addEnumItem("mainQa","Main QA Team").addEnumItem("extDev",
"External Dev Team").addEnumItem("extQa","External QA
Team").allowMultipleValues(true).values(["mainDev","mainQa"]).build())
```

---

12 预定义的 Velocity 宏

有一组预定义的 Velocity 宏可以在 Script Widget 中使用。

纯文本框

```
#message("some plain text")
```

> some plain text

信息消息框

```
#info("something")
```

> Info: something

警告消息框

```
#warning("something")
```

> Warning: something

错误消息框

```
#error("something")
```

> Error: something

加载 CSS

```
#loadCss("http://mywebsite.com/myCSS.css")
```

从 Widget 资源加载 CSS

```
#loadWidgetCss("resources/myCSS.css")
```

加载 JavaScript 文件

```
#loadJs("http://mywebsite.com/myJS.js")
```

从 Widget 资源加载 JavaScript 文件

```
#loadWidgetJs("resources/myJS.js")
```

---

13 在自定义 Widget 中启用 Work Item 属性侧边栏

要启用 Work Item 属性侧边栏：

1. 设置初始 JavaScript 配置，将 Widget 的部分或全部与 Work Item 属性侧边栏关联。这应在渲染过程的开始时完成，通过使用

`com.polarion.alm.shared.api.model.rp.widget.PropertiesSidebarConfiguration` 实现。

2. 设置配置，使 Work Item 属性侧边栏在单击每个单独项目时打开。这是通过使用 `com.polarion.alm.shared.api.model.rp.widget.PropertiesSidebarConfiguredContainer` 将 HTML 属性添加到渲染项目所在的元素来完成的。

示例在 Polarion SDK 示例的 `com.polarion.example.widget` 中提供。

13.1 配置 Work Item 的 HTML 属性以显示 Work Items 侧边栏

项目路径（对于 Work Items：项目 ID/工作项 ID）：

```
com.polarion.alm.shared.api.model.rp.widget.PropertiesSidebarConfiguredContainer.Attributes.referencePath()
```

单击前用于设置样式的 CSS 类：

```
com.polarion.alm.shared.api.model.rp.widget.PropertiesSidebarConfiguredContainer.Attributes.clickableCssClass()
```

侧边栏打开后用于高亮显示项目的 CSS 类：

```
com.polarion.alm.shared.api.model.rp.widget.PropertiesSidebarConfiguredContainer.Attributes.highlightingCssClass()
```

（可选）用于将侧边栏中显示的字段列表与 Work Items 组关联的属性：

```
com.polarion.alm.shared.api.model.rp.widget.PropertiesSidebarConfiguredContainer.Attributes.fieldsConfigurationKey()
```

详情请参阅"13.2 为不同组的 Work Items 显示不同字段"。

可以通过 `com.polarion.alm.shared.api.model.rp.widget.PropertiesSidebarConfiguredContainer.attributes()` 访问这些属性。

13.2 为不同组的 Work Items 显示不同字段

这可以通过使用 `com.polarion.alm.shared.api.model.rp.widget.PropertiesSidebarConfiguration.fieldsConfiguration(Map<String, List<String>>)` 提供初始配置来实现。

示例：

对于 Work Items 看板 Widget（具有包含 User Story 的泳道结构和包含 Implementation Task 或 Issue 的卡片），您可能希望在侧边栏中显示不同的字段集：

例如： - 泳道元素显示：状态、严重性和优先级。 - 卡片元素显示：状态和作者。

为此，在初始配置期间，您需要：

```
Map<String, List<String>> configurationMap = new HashMap<>();
List<String> swimlaneFields = Arrays.asList(new String[] { "status", "severity",
"priority" });
configurationMap.put("swimlane", swimlaneFields);
List<String> cardFields = Arrays.asList(new String[] { "status", "author" });
configurationMap.put("card", cardFields);
context.propertiesSidebarConfiguration().fieldsConfiguration(configurationMap).addT
o(builder);
```

然后在渲染卡片期间：

```
configuredContainer.openSidebarOnClick(tr).reference(wiReference).fieldsConfigurati
onKey("card");
// 单击此项目时，侧边栏将显示 "status", "author"
```

或者在渲染泳道期间：

```
configuredContainer.openSidebarOnClick(tr).reference(wiReference).fieldsConfigurati
onKey("swimlane");
// 单击此项目时，侧边栏将显示 "status", "severity", "priority"
```

> **注意：** 不同的字段列表可以从不同的 Widget 字段参数中读取。

---

14 自定义看板（Kanban）面板

看板面板由按状态列分组的工作项卡片组成。

您可以通过添加自定义 Velocity 代码段来自定义卡片的外观。您可以更改卡片颜色、移除状态边框颜色、显示不同的工作项字段、重新排列已渲染的字段等。自定义 Velocity 脚本将替代默认的 Java 脚本进行渲染。

默认的 Velocity 代码段随 Widget 一起包含在"Board Script"和"Card Script"参数中，是一个很好的起点。（示例代码段开箱即用即可模拟 Java 渲染输出。）

首先，将"Enable Customization"（启用自定义）Widget 参数设置为"Yes"。下面会出现"Board Script"和"Card Script"参数。

Board Script 在看板的开头插入，应始终用于添加自定义 CSS 样式和/或适用于所有卡片的公共 JavaScript。（这避免了为每个卡片重复添加。）

> **警告：** 在此脚本中插入的样式应用于整个页面。

Card Script 定义出现在卡片内的内容。Card Script Velocity 脚本选项允许使用以下额外对象： - `$workItem` - 卡片中渲染的工作项。 - `$columnIndex` - 卡片所在的列。

> **提示：** 启用自定义后，Work Item 属性侧边栏和拖放功能仍然可用。

您始终可以将"Enable Customization"设置为"No"来恢复使用内部 Java 渲染。

以下是一些自定义看板面板卡片外观和内容的示例：

14.1 示例

14.1.1 如何更改卡片颜色

1. 打开包含看板面板的页面，进入编辑模式。
2. 打开"Kanban Board: Parameters"（看板面板：参数）侧边栏。
3. 单击底部的"Advanced"（高级），然后在"Enable Customizations"（启用自定义）下单击"Yes"（是）。
4. 单击"Board Script"（面板脚本）。（按 F11 可全屏查看脚本。按 Esc 退出全屏视图。）
5. 通过在 `.kanbanWorkItemDiv` 中添加 `background-color: #d0d2e0;` 来更改卡片的背景颜色。
6. 通过将 `.kanbanWorkItemTitleGradientDiv` 中的 `background` 属性更改为 `background: linear-gradient(to bottom, rgba(255, 255, 255, 0), #ccffff) repeat scroll 0 0 rgba(0, 0, 0, 0);` 来更改长标题的渐变效果。
7. 通过添加以下内容启用 Work Item 属性侧边栏：

```
.polarion-sidebar-highlight .kanbanWorkItemDiv {
    background-color: #fff9e8;
}
```

默认"Board Script"中唯一的变化是以下示例中的粗体行：

```
...
.kanbanWorkItemDiv {
  padding: 7px 3px 3px 7px;
  box-sizing: border-box;
  border-left: 5px solid;
  border-right: 1px solid #EEEEEE;
  border-top: 1px solid #EEEEEE;
  border-bottom: 1px solid #EEEEEE;
  border-radius: 5px;
  position: relative;
  background-color: #d0d2e0;  /* 更改背景颜色 */
}
...
.kanbanWorkItemTitleGradientDiv {
  bottom: 0px;
  height: 12px;
  left: 0;
  position: absolute;
  width: 100%;
  display: inline-block;
  background: linear-gradient(to bottom, rgba(255, 255, 255, 0), #d0d2e0) repeat
scroll 0 0 rgba(0, 0, 0, 0);
  filter: progid:DXImageTransform.Microsoft.gradient(startColorstr='#00FFFFFF',
endColorstr='#FFFFFFFF', GradientType=0);
 }
...
.polarion-sidebar-highlight .kanbanWorkItemDiv {  /* 启用侧边栏高亮 */
    background-color: #fff9e8;
}
...
```

1. 单击"应用"。

卡片的颜色将更改为所设置的颜色。

（无需对默认 Card Script 进行任何更改。）

14.1.2 如何添加工作项的树结构链接

您可以自定义卡片中显示的内容，在 Card Script 中进行修改。

1. 打开包含看板面板的页面，进入编辑模式。
2. 打开"Kanban Board: Parameters"侧边栏。
3. 单击底部的"Advanced"，然后在"Enable Customizations"下单击"Yes"。
4. 单击"Card Script"。（按 F11 可全屏查看脚本。按 Esc 退出全屏视图。）
5. 将下面的粗体表格文本粘贴到 `<div class="kanbanWorkItemDiv" style="border-left-color: $wiStatusColor;">` 标签下。
6. 单击"应用"。

```
...
<div class="kanbanWorkItemDiv" style="border-left-color: $wiStatusColor;">
 <table width="100%">
 <tbody>
  <tr>
   <td  class="kanbanWorkItemTitleDiv">
    <div class="kanbanWorkItemTitleGradientDiv"></div>
    <a href="$wiLink" target="_top">$workItem.fields().title().render()</a>
   </td>
   #set($treeLink =
$transaction.context().createPortalLink().project("$wiProjectId").workItems().query
("id:$workItem.getReference().id()").toEncodedRelativeUrl())
   #set($suffix = '&tree_depth=2&tab=tree')
   <td style="vertical-align: top;">
    <a href="$treeLink$suffix"  target="_blank">
     <img  style="float: right;"  title="View Item in a Tree Structure" src="/
polarion/ria/images/details/treetable.gif"/>
    </a>
   </td>
   </tr>
        </tbody>
</table>
 <table class="kanbanWorkItemTable">
...
```

一个带有树视图链接的树图标已添加到卡片中。

15 自定义快速创建（Quick Create）对话框的内容

快速创建对话框可通过添加到 Polarion 导航标题中的"Create..."按钮访问。其内容已预配置，但可以自定义用户可以创建的工件。

默认配置包含此对话框当前可用的大多数功能，可用作创建自己配置的初始模板。

您可以从 Polarion 帮助（Configure Quick Create）下载默认配置页面存档，并根据您的需求进行自定义。将自定义页面导入 Polarion，并将其路径定义为快速创建对话框的源。

快速创建对话框不适用于项目组（Project Group）、基线（Baseline）或管理（Administration）上下文。

这里我们将介绍我们在默认快速创建配置中包含的功能。您还可以在我们的 JavaDoc 中找到所用 API 的详细描述：`JsActionsBuilder`。

`[POLARION_HOME]/polarion/sdk/doc/javadoc-rendering/com/polarion/alm/shared/api/utils/js/JsActionsBuilder.html`

快速创建中可用的所有操作可以分为上下文无关操作和上下文相关操作。

15.1 上下文无关操作（Context-independent actions）

这些操作无论用户在 Polarion UI 中的什么位置都可在快速创建对话框中使用。

所有操作分为三列：Work Item（工作项）、Document（文档）和 Other（其他）。

15.1.1 Work Item（工作项）

Work Item 列包含为当前项目配置的所有工作项类型的列表。

每行可单击，工具提示包含工作项类型描述（取自"管理 - 工作项 - 类型"），或仅包含工作项类型标签（如果未定义描述），鼠标悬停时可见。当前配置允许您禁用基于权限拒绝用户的操作。

当用户单击项目时，将打开相应类型的新工作项表单。（然后快速创建对话框关闭。）

在项目上下文中，用户可以创建项目中配置的每种工作项类型的工作项，使用 `jsBuilder.createWorkItem()` 并在输入中发送类型：

```
<div class="polarion-quick-create-column">
#foreach($type in $types)

#if($transaction.workItems().can().inScope().forProject($projectId).createWorkItemO
fType($type.id))
   #set($itemDisabled="")
   #set($tooltip = "" )
 #else
   #set($itemDisabled="polarion-quick-create-column-item-disabled")
   #set($tooltip=$loc.getString('js.actions.createWorkItem.no.permission'))
 #end
 <div class="polarion-quick-create-column-item $itemDisabled"
onclick='$jsBuilder.createWorkItem().type($type.id)'>
   $type.render().withCustomTooltip($tooltip)
 </div>
 #end
</div>
```

在全局上下文中，用户可以调用创建工作项对话框，需要选择项目和类型才能继续创建。 使用不带类型规范的 `jsBuilder.createWorkItem()`：

```
<div class="polarion-quick-create-column-item"
onclick='$jsBuilder.createWorkItem()'>
   <img class="polarion-quick-create-column-item-img" src="/polarion/icons/default/
actions/new_workitem.gif">
        $loc.getString('navigation.quickCreateDialogCreateWorkItem')
</div>
```

15.1.2 Document（文档）

项目上下文中的文档列包含： - 导入 Word，通过调用 `jsBuilder.importWordDocument()` 实现 - 导入 ReqIF 文档，通过调用 `jsBuilder.importReqIfDocument()` 实现 - 在当前项目中为每个配置的文档类型创建文档，通过调用 `jsBuilder.createDocument()` 并在输入中发送类型实现

每行可单击，工具提示包含文档类型的描述（取自"管理 - 文档和页面 - 文档类型"），或仅包含文档类型标签（如果未定义描述），鼠标悬停时可见。当前配置允许您禁用基于权限拒绝用户的操作。

通过单击具有所需文档类型的行，将打开"创建新文档"对话框，其中预定义了该文档类型，如果用户在调用快速创建对话框中的创建文档操作时位于 Polarion 的空间上下文中，还会预定义空间：

```
<div class="polarion-quick-create-column">
       #if(!
$transaction.documents().can().inScope().forProject($projectId).create())
           #set($itemDisabled="polarion-quick-create-column-item-disabled")
           #set($tooltip=$loc.getString('js.actions.createDocument.no.permission'))
       #end
       <div class="polarion-quick-create-column-item $itemDisabled"
       onclick='$jsBuilder.importWordDocument()' title="$tooltip">
          <img class="polarion-quick-create-column-item-img"
src="/polarion/ria/images/dle/operations/actionMsWordRoundtrip16.svg">
          $loc.getString('navigation.quickCreateDialogCustomImportWord')
       </div>
       <div class="polarion-quick-create-column-item $itemDisabled"
       onclick='$jsBuilder.importReqIfDocument()' title="$tooltip">
           <img class="polarion-quick-create-column-item-img" src="/polarion/ria/
images/control/Reqif.png">
           $loc.getString('navigation.quickCreateDialogCustomImportReqIF')
       </div>
       <div class="polarion-quick-create-column-item-separator"></div>
           #foreach($type in $types)
               <div class="polarion-quick-create-column-item polarion-ellipsis
$itemDisabled"
               onclick='$jsBuilder.createDocument().type($type.id)'>
                  $type.render().withCustomTooltip("$tooltip")
               </div>
           #end
</div>
```

在全局上下文中，用户无法处理文档。（这就是为什么默认配置仅包含向用户显示此信息的消息：）

```
<div style="display: block; padding:
6px;">$loc.getString('navigation.quickCreateDialogDocumentsCannotCreate')</div>
```

15.1.3 Other（其他）

对于全局和项目上下文，用户都可以创建富文本页面（Rich Pages）和空间（Spaces）。

为 LiveReport 和信息页面（Info Pages）渲染了单独的操作。 - `jsBuilder.createRichPage()` 用于创建 LiveReport 页面。 - `jsBuilder.createRichPage().liveReport(false)` 用于创建信息页面。

区别仅在于使用的页面模板。（这与从空间索引页面可用的"创建新工件"对话框的原理相同。） - `jsBuilder.createSpace()` 用于创建空间。

对于所有操作，都会考虑当前用户上下文： 如果用户在 Polarion 中的任何空间上下文中，该空间将用作新富文本页面的目标空间或新空间的父空间。

```
<div class="polarion-quick-create-column-item $itemDisabledPage"
onclick='$jsBuilder.createRichPage()' title="$tooltipReport">
  <img class="polarion-quick-create-column-item-img" src="/polarion/ria/images/
topicIconsSmallDark/richpage.png">
  $loc.getString('dialog.createNewArtifact.newReport.title')
</div>
<div class="polarion-quick-create-column-item $itemDisabledPage"
onclick='$jsBuilder.createRichPage().liveReport(false)' title="$tooltipInfo">
  <img class="polarion-quick-create-column-item-img" src='/polarion/ria/images/
topicIconsSmallDark/richpage.png'>
  $loc.getString('dialog.createNewArtifact.newTextPage.title')
</div>
<div class="polarion-quick-create-column-item $itemDisabledPage"
onclick='$jsBuilder.createSpace()' title="$tooltip">
  <img class="polarion-quick-create-column-item-img" src="/polarion/ria/images/
topicIconsSmallDark/space.png">
  $loc.getString('definition.space')
</div>
```

如果用户在项目上下文中，可以使用 `jsBuilder.createPlan()`、`jsBuilder.createTestRun()`、`jsBuilder.createCollection()` 创建计划（Plans）、测试运行（Test Runs）和集合（Collections）：

```
<div class="polarion-quick-create-column-item-separator"></div>
  #set($tooltipPlan="")
  #if(!$transaction.plans().can().inScope().forProject($projectId).create())
     #set($itemDisabledPlan="polarion-quick-create-column-item-disabled")
     #set($tooltipPlan=$loc.getString('js.actions.createPlan.no.permission'))
  #end
  <div class="polarion-quick-create-column-item $itemDisabledPlan"
onclick='$jsBuilder.createPlan()'
title="$tooltipPlan">
     <i class="fa fa-bar-chart-o polarion-font-icon-16 polarion-quick-create-
column-item-img" ></i>
     $loc.getString('definition.plan')
  </div>

  #set($tooltipTr="")
  #if(!$transaction.testRuns().can().inScope().forProject($projectId).create())
     #set($itemDisabledTr="polarion-quick-create-column-item-disabled")
     #set($tooltipTr=$loc.getString('js.actions.createTestRun.no.permission'))
  #end
  <div class="polarion-quick-create-column-item $itemDisabledTr"
onclick='$jsBuilder.createTestRun()'
title="$tooltipTr">
     <img class="polarion-quick-create-column-item-img" src="/polarion/ria/images/
topicIconsSmallDark/testruns.svg">
     $loc.getString('definition.testRun')
  </div>

  #set($tooltipColl="")
  #if(!
$transaction.baselineCollections().can().inScope().forProject($projectId).create())
     #set($itemDisabledColl="polarion-quick-create-column-item-disabled")
     #set($tooltipColl=$loc.getString('js.actions.createCollection.no.permission'))
  #end
  <div class="polarion-quick-create-column-item $itemDisabledColl"
onclick='$jsBuilder.createCollection()'
title="$tooltipColl">
     <img class="polarion-quick-create-column-item-img"
src="/polarion/ria/images/topicIconsSmallDark/collectionsTopic.svg">
     $loc.getString('definition.collection')
  </div>
  #end
</div>
```

15.2 上下文相关操作（Context-dependent actions）

您可以使用 `$renderingContext.getDisplayedReference()` 向 Velocity 代码提供当前用户的上下文。

基于此信息，可以在用户在 Polarion 上下文中定义时才在快速创建中渲染操作。

在默认配置中，我们有当用户选择工作项或处于 LiveDoc 文档上下文时可用操作的示例。

15.2.1 对于选中的项目（For Selected Item）

对于"选中的工作项"条件，以下操作可用：创建同级工作项（Create Sibling Work Item）、创建工作子项（Create Child Work Item）。

您的用户在以下位置工作时可以为选中的工作项创建同级或子工作项： - 当他们在工作项跟踪器的表/树视图中选择属于文档的单个工作项时。 - 当他们在处理文档时在表/树视图中选择单个工作项时。 - 如果他们正在处理工作项表单（编辑器）中的文档工作项。 - 如果选中的工作项不是引用工作项。（当前 API 仅在其主文档中创建链接工作项。对于引用工作项，该操作被禁用。） - 如果文档不是历史修订版。

```
#if($selectedWorkItems.size() == 1)
    #set($selectedWorkItemReference=$selectedWorkItems.iterator().next())
#elseif($displayedReference && $displayedReference.prototype().name()=="WorkItem")
    #set($selectedWorkItemReference=$displayedReference)
    #set($isFullscreenWorkItem=true)
#end
```

```
#if($selectedWorkItemReference)
    #set($selectedWorkItem=$selectedWorkItemReference.get($transaction))
    #set($document=$selectedWorkItem.fields().module().getIfCan())
    #set($isHistoricalRevision=$displayedReference.requestedRevision())

#set($elementIsInternal=$selectedObject.items().isInternal($selectedWorkItem))

#if($document && ("home"!=$renderingContext.urlParameters().get("tab") ||
$isFullscreenWorkItem) && !$isHistoricalRevision &&
($elementIsInternal==true || "!elementIsInternal"==""))
    #set($documentAllowedTypes=$document.fields().allowedWITypes().toArrayList())
```

如果所有条件都满足，用户将在单独的"对于选中的项目"标题下看到以下上下文操作： 创建工作同级项（Create Sibling Work Item）和创建工作子项（Create Child Work Item）。

结果，项目将作为链接项目在相同的文档上下文中创建到选中的工作项。

用户可以选择结果工作项的类型。在默认配置示例中有不同的渲染逻辑： - 如果在文档工作项呈现中只定义了一个工作项类型，上下文操作按钮覆盖整行。 - 如果在文档工作项呈现中定义了最多四个工作项类型，则渲染带有工作项类型图标和扩展工具提示的小按钮。 - 默认配置不渲染超过文档工作项呈现中定义的前四个工作项类型。

您可以根据需要更新配置。（这只是一个示例。）

```
  <div class="polarion-quick-create-separator-with-icons">
      <img src="/polarion/ria/images/actions/slibing_wi.gif">同级（Sibling）
  </div>
 #if($documentAllowedTypes.size() > 1)
            #foreach($type in $documentAllowedTypes)

#if($transaction.workItems().can().inScope().forProject($projectId).createWorkItemO
fType($type.id))
                    #set($itemDisabled="")
                    #set($tooltip="")
                    #set($customTooltip=" ${type.label} (in this Document) -
$type.fields.description.getIfCan() ")
                #else
                    #set($itemDisabled="polarion-quick-create-column-item-
disabled")

#set($tooltip=$loc.getString('js.actions.createWorkItem.no.permission'))

#set($customTooltip=$loc.getString('js.actions.createWorkItem.no.permission'))
                #end
                #if($velocityCount < 5)
                    <div class="polarion-quick-create-context-aware-buttons
$itemDisabled"

onclick='$jsBuilder.createLinkedWorkItem().type($type.id).linkedType("sibling")'>

$type.render().withText(false).withCustomTooltip($customTooltip)
                    </div>
                #end
            #end
        #elseif($documentAllowedTypes.size() == 1)
            #foreach($type in $documentAllowedTypes)

#if($transaction.workItems().can().inScope().forProject($projectId).createWorkItemO
fType($type.id))
                    #set($itemDisabled="")
                    #set($tooltip="")
                #else
                    #set($itemDisabled="polarion-quick-create-column-item-
disabled")

#set($tooltip=$loc.getString('js.actions.createWorkItem.no.permission'))
                #end
                <div class="polarion-quick-create-context-aware-button
$itemDisabled"
```

```
onclick='$jsBuilder.createLinkedWorkItem().type($type.id).linkedType("sibling")'>
                    $type.render().withCustomTooltip($tooltip)
                </div>
            #end
        #end
        <div class="polarion-quick-create-separator-with-icons">
            <img src="/polarion/ria/images/actions/linking_wi.gif">子项（Child）
        </div>
        ... 与上面"同级"相同的 Velocity 代码
```

15.2.2 对于此文档（For this Document）

如果用户处于任何文档上下文中（无论使用什么视图：文档编辑器、表或树），在快速创建对话框中的单独"对于此文档"标题下将提供额外的操作： - 为当前文档创建文档基线（通过 `jsBuilder.createBaseline()` ） - 复用当前文档（通过 `jsBuilder.duplicate()` ） - 分支当前文档（通过 `jsBuilder.branchDocument()` ）

检查当前用户上下文的方式：
`#if($renderingContext.getDisplayedReference().prototype().name()=="Document")` 。

```
<div class="polarion-quick-create-context-aware-button $itemDisabled"
     onclick='$jsBuilder.createBaseline()' title="$tooltip">
         <img class="polarion-quick-create-context-document-img" src="/polarion/
ria/images/dle/create_baseline.png">
         $loc.getString('navigation.quickCreateDialogCustomDocumentBaseline')
</div>
<div class="polarion-quick-create-context-aware-button $itemDisabled"
   onclick='$jsBuilder.duplicate()' title="$tooltip">
       <img class="polarion-quick-create-context-document-img" src="/polarion/ria/
images/dle/operations/reuse.png">
        $loc.getString('navigation.quickCreateDialogCustomDocumentReuse')
</div>
<div class="polarion-quick-create-context-aware-button $itemDisabled"
onclick='$jsBuilder.branchDocument()' title="$tooltip">
       <img src="/polarion/ria/images/dle/operations/branch.png">
        $loc.getString('navigation.quickCreateDialogCustomDocumentBranch')
</div>
```

---

16 常见问题（F.A.Q）

问：如何获取当前项目的 ID？ 答：在 Velocity 中： `$page.reference.projectId`

问：导出为 PDF 时 HTML 表格丢失样式和/或没有列怎么办？ 答：PDF 导出要求 HTML 标签 `<tr>`、`<th>` 和 `<td>` 必须用 `</tr>`、`</th>` 和 `</td>` 关闭。